Problema: CSES 1668 – Building Teams

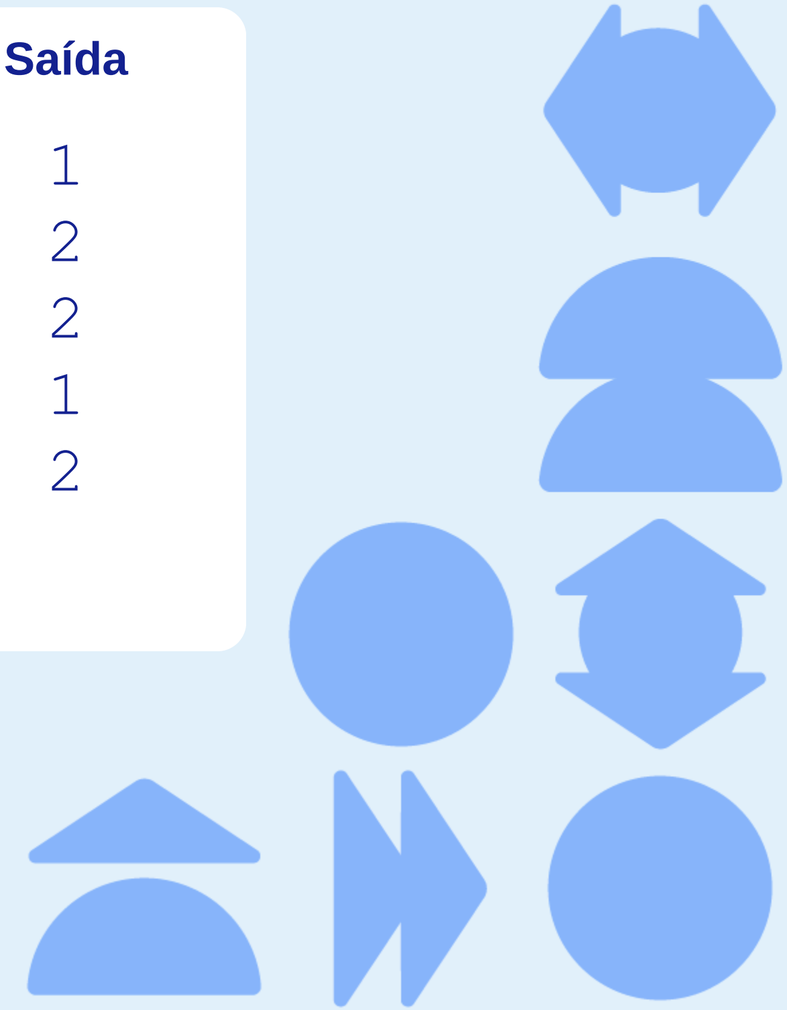
O QUE O PROBLEMA PEDE

n alunos e **m** amizades.
Dividir todos em dois times, sem que dois amigos fiquem no mesmo time.
Saída: o time (1 ou 2) de cada aluno ou **IMPOSSIBLE**.

Restrições $n \le 10^5$ · $m \le 2 \cdot 10^5$

EXEMPLO

Entrada	Saída
5 3 1 2 1 3 4 5	1 2 2 1 2



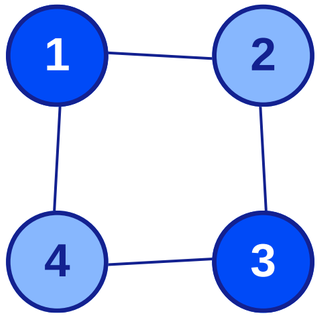
Raphael Albuquerque (2413093) · Vinicius Freitas (2422972)
Orientador: Prof. Me. Ricardo Carubbi

Modelagem e classificação do grafo

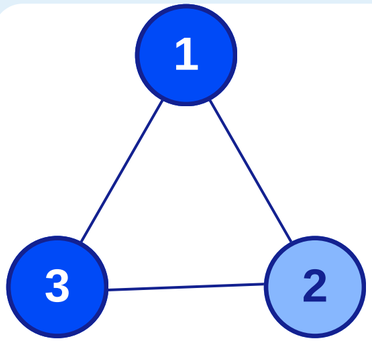
Vértices	alunos (1 … n)
Arestas	amizades (u, v)
Tipo	não dirigido e simples
Conectividade	pode ser desconexo
Representação	lista de adjacência

Propriedade: grafo bipartido
possui 2-coloração ⇔ não tem ciclo ímpar

CRITÉRIO



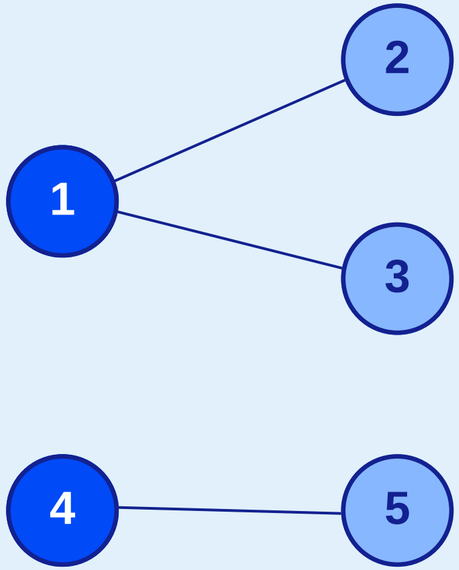
ciclo par
✔ bipartido



ciclo ímpar
✘ IMPOSSIBLE

Instância pequena e rastreamento

GRAFO



Lista de adjacência
1→2,3 · 2→1 · 3→1
4→5 · 5→4

RASTREAMENTO DA DFS (PILHA EXPLÍCITA)

Passo	Ação	Pilha	equipe
1	início em 1 → equipe 1	[1]	1 0 0 0 0
2	v=1, w=2 → equipe 2	[1,2]	1 2 0 0 0
3	2 sem vizinhos → pop	[1]	1 2 0 0 0
4	v=1, w=3 → equipe 2	[1,3]	1 2 2 0 0
5	pop 3, pop 1	[ ]	1 2 2 0 0
6	início em 4 → 1; 5 → 2	[4,5]	1 2 2 1 2
7	pop 5, pop 4 → fim	[ ]	1 2 2 1 2

Saída: 1 2 2 1 2

```java
class NonrecursiveDFS {
    private final int[] equipe;
    private boolean impossivel = false;

    @SuppressWarnings("unchecked")
    public NonrecursiveDFS(Graph G) {
        int V = G.V();
        equipe = new int[V];

        Iterator<Integer>[] adj = (Iterator<Integer>[]) new Iterator[V];
        for (int v = 0; v < V; v++) {
            adj[v] = G.adj(v).iterator();
        }

        Deque<Integer> stack = new ArrayDeque<>();

        for (int i = 0; i < V; i++) {
            if (equipe[i] == 0) {
                equipe[i] = 1;
                stack.push(i);

                while (!stack.isEmpty()) {
                    int v = stack.peek();

                    if (adj[v].hasNext()) {
                        int w = adj[v].next();

                        if (equipe[w] == 0) {
                            equipe[w] = 3 - equipe[v];
                            stack.push(w);
                        } else if (equipe[w] == equipe[v]) {
                            impossivel = true;
                            return;
                        }
                    } else {
                        stack.pop();
```

```java
public class Main {
    public static void main(String[] args) {
        FastScanner scanner = new FastScanner();
        String nStr = scanner.next();
        if (nStr == null) return;

        int n = Integer.parseInt(nStr);
        int m = scanner.nextInt();

        Graph G = new Graph(n);

        for (int i = 0; i < m; i++) {
            int u = scanner.nextInt() - 1;
            int v = scanner.nextInt() - 1;
            G.addEdge(u, v);
        }

        NonrecursiveDFS dfs = new NonrecursiveDFS(G);

        if (dfs.isImpossivel()) {
            System.out.println("IMPOSSIBLE");
        } else {
            StringBuilder sb = new StringBuilder();
            int[] equipes = dfs.getEquipes();
            for (int i = 0; i < n; i++) {
                sb.append(equipes[i]).append(i == n - 1 ? "" : " ");
            }
            System.out.println(sb.toString());
```

Complexidade e casos especiais

O(n + m)

Tempo

cada vértice e cada aresta visitados uma vez

O(n + m)

Grafo

lista de adjacência

O(n)

Memória auxiliar

vetor equipe[] + pilha

CASOS ESPECIAIS

- **Grafo desconexo:** nova DFS a cada vértice sem equipe.
- **Ciclo ímpar:** vizinho com a mesma equipe ⇒ IMPOSSIBLE.
- **n até 10^5:** pilha explícita evita estouro de recursão.

Conclusão: bipartido ⇔ a DFS 2-colore sem conflito.

